Spring Security 参考手册

翻译 @miwenshu

目錄

1. Introduction
2. 前言
3. 入门
    i. 简介
    ii. Spring Security 4 新特性
    iii. Java配置
        i. 基础的网站安全 Java配置
        ii. HttpSecurity
        iii. Java配置和表单登录
        iv. 验证请求
        v. 处理登出
        vi. 验证
        vii. 多个HttpSecurity
        viii. 方法安全
        ix. 已配置对象的后续处理
    iv. 安全命名空间配置
        i. 命名空间的设计
        ii. 开始使用安全命名空间配置
        iii. 高级网站功能
        iv. 方法安全
        v. 默认的访问决策管理器
        vi. 验证管理器和命名空间
    v. 应用程序示例
    vi. Spring Security 社区
4. 架构和实现
    i. 运行时环境
    ii. 技术预览
    iii. 核心组件
    iv. new
5. Web应用安全

Spring Security 参考手册

作者

Ben Alex , Luke Taylor , Rob Winch , Gunnar Hillert

4.0.2.RELEASE

Copyright © 2004-2015 Copies of this document may be made for your own use and for distribution to others, provided that you do not charge any fee for such copies and further provided that each copy contains this Copyright Notice, whether distributed in print or electronically.

英文原版

http://docs.spring.io/spring-security/site/docs/current/reference/htmlsingle/#what-is-acegi-security

翻译

学习Spring 顺便翻译了。翻译有不当的地方请见谅。

Vincent Mi http://vnzmi.com
miwenshu#gmail.com

前言

Spring Security 为基于Java EE的企业软件应用程序提供一个全面的解决方案。正如你将从这个参考指南发现的，我们试图为你提供一个有用的并且高度可配置的安全系统。

安全是一个不断移动的目标，采取一个全面的全系统的方法很重要。在安全领域，我们鼓励你采取"layers of security""(安全层)，这样每一层尽可能的在自己范围内保证安全，连续的层提供额外的安全性。安全层更密集你的程序将更加健壮更加安全。在最底层，你需要处理传输安全和系统识别这些问题，以减少中间人攻击。接下来，您将通常利用防火墙，或许使用VPN或者IP担保以确保只有授权的系统能够尝试连接。在企业环境中你可以部署一个DMZ将面向公众的服务器和数据库以及应用服务器分隔开来。你的操作系统也将扮演重要的部分，解决问题，类似，使用非特权用户运行进程和提高文件系统安全性。操作系统通常会配置自己的防火墙。但愿在某处前进的道路上，你会试图阻止拒绝服务攻击和暴力攻击。一个入侵检测系统将在监视和响应攻击时非常有用，这种系统能采取保护动作，比如实时阻断违规的TCP/IP地址。在更高的层，你的Java虚拟机希望被配置为尽量减少不同的Java类型授予的权限，然后将您的应用程序将增加其自身的指定域特定的安全配置。Spring Security 使后者，应用程序安全更加容易。

当然你需要妥善处理上面提到的所有安全层，连同各层的管理因素。这样的管理因素大体包括安全公告监测、补丁、人员审查、审计、变更控制、工程管理系统、数据备份、灾难恢复、性能基准测试、负载监控、集中式日志记录、事件响应程序等。

Spring Security 致力于在企业应用程序安全层对你进行帮助，你会发现这里有如此不同的需求正如业务问题的领域。一个银行应用程序具有与电子商务应用不同的需求。电子商务应用程序同企业销售自动化工具有不同的需求。这些定制需求使得应用安全有趣、有挑战性和有回报。

请阅读第二部分，从“入门”开始。这一章节将为您介绍的框架和基于命名空间的配置系统，你可以配置好应用，很快运行起来。阅读第三部分“架构和实现”，来了解Spring Security如何工作和一些你可能需要使用的类。本指南的其余部分是更传统的引用样式结构，设计用于按需进行阅读。我们也建议您尽可能阅读完一般的应用安全问题。Spring Security不是万能的，不能解决所有安全问题。重要的是在应用设计开始之初就考虑到安全性。后期改造不是一个好的主意。特别是，如果你正在构建一个Web应用程序，你应该知道许多潜在的漏洞，比如：跨站脚本、请求伪造和会话劫持。这些你已开始就应该考虑。这个网站 OWASP 维护了一个10大网站应用漏洞列表和一些有用的参考信息。

我们希望这个参考手册对你来说比较有用，欢迎你的反馈和建议。

最后欢迎你来到Spring Security 社区。

入门

本指南的稍后章节会对框架的架构和实现类进行一个深度的讨论，如果你想对Spring Security进行一个深度定制，这一章节将会包含包含你需要了解的内容。在本章我们将会介绍Spring Security 3.0 ,给项目的历史进行简要的概述，简单的讲讲如何开始使用这个框架。尤其是我们将看看命名空间配置，他提供与传统Spring Bean你必须连接所有实现类的途径更简单的方式保护你的应用程序。

我们也会看看示例应用。在你阅读后面的章节之前你值得试着运行并体验它。当你对框架了解更多的时候你还可以回过头来回顾一下。同时请参阅项目的网站，因为他有创建这个项目的有用的信息，以及文章、视频和教程。

简介

Spring Security是什么?

Spring Security提供了基于Java EE的企业应用软件全面的安全服务。这里特别强调支持使用Spring框架构建的项目，Spring框架是企业软件开发Java EE方案的领导者 . 如果你还没有使用Spring来开发企业应用程序，我们热忱的鼓励你仔细的看一看。熟悉Spring特别是依赖注入原理将帮助你更快更方便的使用Spring Secirity.

人们使用Spring Security的原因有很多，但大部分都发现了Java EE的Servlet规范或EJB规范中的安全功能缺乏典型企业应用场景所需的深度。提到这些规范，重要的是要认识到他们在WAR或EAR级别无法移植。因此如果你更换服务器环境，这里有典型的大量工作去重新配置你的应用程序员安全到新的目标环境。使用Spring Security解决了这些问题，也为你提供许多其他有用的，可定制的安全功能。

正如你可能知道的两个应用程序的两个主要区域是“认证”和“授权”（或者访问控制）。这两个主要区域是Spring Security的两个目标。“认证”，是建立一个他声称的主体的过程（一个“主体”一般是指用户，设备或一些可以在你的应用程序中执行动作的其他系统）。“授权”指确定一个主体是否允许在你的应用程序执行一个动作的过程。为了抵达需要授权的点，主体的身份已经有认证过程建立。这个概念是通用的而不只在Spring Security中。

在身份验证层，Spring Security的支持多种认证模式。这些验证模型绝大多数都是要么由第三方提供，或由相关的标准组织，如互联网工程任务组开发。另外 Spring Security 提供自己的一组认证功能。具体而言，Spring Security目前支持所有这些技术集成的身份验证：

- HTTP BASIC 认证头(基于 IETF RFC-based 标准)
- HTTP Digest 认证头 (基于 IETF RFC-based 标准)
- HTTP X.509 客户端证书交换 (IETF RFC-based 标准)
- LDAP (一个非常常见的方法来跨平台认证需要，尤其是在大型环境)
- Form-based authentication (用于简单的用户界面)
- OpenID 认证
- Authentication based on pre-established request headers (such as Computer - Associates Siteminder)根据预先建立的请求头进行验证
- JA-SIG Central Authentication Service (CAS, 一个开源的SSO系统)
- Transparent authentication context propagation for Remote Method Invocation (RMI) and HttpInvoker (Spring远程协议)
- Automatic "remember-me" authentication (所以你可以勾选一个框以避免预定的时间段再认证)
- Anonymous authentication (让每一个未经验证的访问自动假设为一个特定的安全标识)
- Run-as authentication (在一个访问应该使用不同的安全标识时非常有用)
- Java Authentication and Authorization Service (JAAS)
- JEE container autentication (所以如果愿意你可以任然使用容器管理的认证s)
- Kerberos
- Java Open Source Single Sign On (JOSSO) *
- OpenNMS Network Management Platform *
- AppFuse *
- AndroMDA *
- Mule ESB *
- Direct Web Request (DWR) *
- Grails *
- Tapestry *
- JTrac *
- Jasypt *
- Roller *
- Elastic Path *
- Atlassian Crowd *

- 你自己的认证 (看下面)

*表示由第三方提供

很多独立软件供应商，因为灵活的身份验证模式而选择Spring Security。这样做允许他们快速的集成到他们的终端客户需求的解决方案而不用进行大量工程或者改变客户的环境。如果上面的验证机制不符合你的需求，Spring Security 是一个开放的平台，要实现你自己的验证机制非常检查。Spring Security的许多企业用户需要与不遵循任何特定安全标准的“遗留”系统集成，Spring Security可以很好的与这类系统集成。

无论何种身份验证机制， Spring Security提供一套的授权功能。这里有三个主要的热点区域，授权web请求、授权方法是否可以被调用和授权访问单个域对象的实例。为了帮助让你分别了解这些差异，认识在Servlet规范网络模式安全的授权功能，EJB容器管理的安全性和文件系统的安全。Spring Security在这些重要的区域提供授权功能，我们将在手册的后面进行介绍。

历史

Spring Security以“The Acegi Security System for Spring”的名字始于2013年晚些时候。一个问题提交到Spring开发者的邮件列表，询问是否已经有考虑一个基于Spring的安全性实现。那时候Spring 的社区相对较小（相对现在）。实际上Spring自己在2013年只是一个存在于SourceForge中的项目，这个问题的回答是，这是一个值得研究的领域，虽然目前时间的缺乏阻止了我们对它的探索。

考虑到这一点，一个简单的安全实现建成但是并没有发布。几周后，Spring社区的其他成员询问了安全性，这次这个代码被发送给他们。其他几个请求也跟随而来。到2014年1月大约有20人在使用这个代码。 . These pioneering users were joined by others who suggested a SourceForge project was in order, which was duly established in March 2004.

在早些时候，这个项目没有任何自己的验证模块，身份验证过程依赖于容器管理的安全性和Acegi安全性。而不是专注于授权。开始的时候这很适合，但是越来越多的用户请求额外的容器支持。 the fundamental limitation of container-specific authentication realm interfaces became clear. There was also a related issue of adding new JARs to the container’s classpath, which was a common source of end user confusion and misconfiguration.

Acegi Security-specific authentication services were subsequently introduced. Around a year later, Acegi Security became an official Spring Framework subproject. The 1.0.0 final release was published in May 2006 - after more than two and a half years of active use in numerous production software projects and many hundreds of improvements and community contributions.

Acegi Security became an official Spring Portfolio project towards the end of 2007 and was rebranded as "Spring Security".

Today Spring Security enjoys a strong and active open source community. There are thousands of messages about Spring Security on the support forums. There is an active core of developers who work on the code itself and an active community which also regularly share patches and support their peers.

发布版本号

了解Spring Security 发布版本号如何工作是很有用的，他可以帮助你识别出工作（或缺乏的功能）涉及到参与迁移到项目的未来版本，每个发布使用3个整数，MAJOR.MINOR.PATCH(主版本.次要版本.补丁版本). 这样做的目的是主版本是不兼容的，API大范围的升级。次要版本应该保留大部分源代码和二进制兼容旧版本的次要版本，认为可能有一些设计变更和不兼容的更新。补丁版本应该向前向后完美兼容.包含一些bug和缺陷修复这些意外的改变.

在某种程度上，你受到变化的影响取决于你的代码是如何紧密集成的。如果你正在做大量定制，你更可能受到比简单的命名空间配置更大的影响。

你应该总是推出一个新版本之前彻底测试你的应用程序。

获取 Spring Security

你可以通过几种方式获取Spring Security。 你可以从Spring Security页面下载一个分发包。从Maven库下载分离的jar文件。另外你也可以从源代码自己编译。

使用Maven

一个最小的Spring Security Maven依赖通常和下面的的类似:

pom.xml.

```
<dependencies>
<!-- ... 其他依赖元素.. -->
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-web</artifactId>
    <version>4.0.2.RELEASE</version>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-config</artifactId>
    <version>4.0.2.RELEASE</version>
</dependency>
</dependencies>
```

如果你使用了额外的功能比如 LDAP,OpenID,等等，你需要包含适当的模块， 查阅 Section 1.4.3, “Project Modules”.

Maven 仓库

所有GA发布版本 (版本号以 .RELEASE结尾) 都被部署到Maven Central, 所以不需要在你的 pom里设置额外的库

如果你使用了一个 SNAPSHOT 版本, 你需要确认你设置了Snapshot库，如下:

pom.xml.

```
<repositories>
<!-- ... 其他仓库元素 ... -->
<repository>
    <id>spring-snapshot</id>
    <name>Spring Snapshot Repository</name>
    <url>http://repo.springsource.org/snapshot</url>
</repository>
</repositories>
```

如果你正在使用一个里程碑或者发布候选版本，你需要确保你所定义的Spring里程碑库，如下图所示：

pom.xml.

```
<repositories>
<!-- ... 其他仓库元素 ... -->
<repository>
    <id>spring-milestone</id>
    <name>Spring Milestone Repository</name>
    <url>http://repo.springsource.org/milestone</url>
</repository>
</repositories>
```

Spring 框架 Bom

Spring Security是针对Spring框架 4.1.6.RELEASE建立的，但是应该快要与4.0.x正常工作。很多用户都会有这个问题，Spring Security传递依赖解析到Spring框架4.1.6.RELEASE这可能会导致奇怪的classpath问题。

规避这个问题的一种方式是在你的pom文件的 `<depencyManagement>` 部分包含所有的Spring框架模块。另一种方法是像下面这样在 `pom.xml` 中包含 `spring-framework-bom`：

pom.xml.

```
<dependencyManagement>
    <dependencies>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-framework-bom</artifactId>
        <version>4.1.6.RELEASE</version>
        <type>pom</type>
        <scope>import</scope>
    </dependency>
    </dependencies>
</dependencyManagement>
```

这样将确保Spring Security传递的所有依赖都使用Spring 4.16.RELEASE的模块。

> 这种方式使用了Maven的 "bill of materials" (BOM) 概念， Maven 2.0.9+可用.如果想了解Maven如何解析依赖请参考Maven的依赖机制文档的介绍。

使用Gradle

最小的Gradle一组典型的依赖如下:

build.gradle.

```
dependencies {
    compile 'org.springframework.security:spring-security-web:4.0.2.RELEASE'
    compile 'org.springframework.security:spring-security-config:4.0.2.RELEASE'
}
```

如果你使用了LDAP,OpenID等等，你需要包含相应的模块，参阅 1.4.3, “Project Modules”.

Gradle 仓库

所有的GA发布版本（以.RELEASE结尾）都部署到了Maven Central库，所以使用mavenCentral()就足够了。

build.gradle.

```
repositories {
    mavenCentral()
}
```

如果你正在使用了SNAPSHOT 版本, 你需要确保你使用了Spring的Snapshot库，定义如下:

build.gradle.

```
repositories {
    maven { url 'https://repo.spring.io/snapshot' }
}
```

如果你正在使用里程碑或者发布候选版。你需要确认你使用了Spring的里程碑仓库，定义如下:

build.gradle.

```
repositories {
    maven { url 'https://repo.spring.io/milestone' }
}
```

使用Spring 4.0.x 和Gradle

解析依赖传递时Gradle默认会使用最新的版本。这意味着当在Spring 4.1.6.RELEASE下使用Spring Security 4.0.2.RELEASE我们不需要做额外的工作。然而有时出现一些问题，最好使用Gradle的ResolutionStrategy 按照下面的方式来处理：

build.gradle.

```
configurations.all {
    resolutionStrategy.eachDependency { DependencyResolveDetails details ->
        if (details.requested.group == 'org.springframework') {
            details.useVersion '4.1.6.RELEASE'
        }
    }
}
```

这样将确保所有Spring Security传递的依赖使用Spring 4.1.6.RELEASE 的模块.

> 这个例子使用Gradle 1.9，但可能需要在gradle的未来版本中进行修改，因为这是在一个Gradle的孵化功能。

项目模块

在Spring Security 3.0,代码库被分割到单独的jar，这样可以更清楚的分隔不同功能区域和第三方依赖。如果你使用Maven构建你的项目，那么这些都是需要你添加到你的pom.xml中的。甚至如果你没有使用Maven,我们建议你请求pom.xml文件来获取第三方依赖和版本。另一个好办法是检查示例应用程序的库。

核心模块 spring-security-core.jar

包含核心的验证和访问控制类和接口，远程支持和基本的配置API.任何使用Spring Security的应用程序都需要这个模块。支持独立应用程序、远程客户端、服务层方法安全和JDBC用户配置。包含以下顶层包：

```
org.springframework.security.core
org.springframework.security.access
org.springframework.security.authentication
org.springframework.security.provisioning
```

远程调用 - spring-security-remoting.jar

提供与Spring Remoting的集成. 通常你不需要这个模块，除非你正使用Spring Remoting编写远程客户端。主要的包是 `org.springframework.security.remoting`.

网页 spring-security-web.jar

包括过滤器和网站安全相关的基础代码。使用Servlet API的任何应用依赖他。如果你需要Spring Security网页验证服务和基于URL的访问控制你需要这个模块。主包名为 `org.springframework.security.web`.

配置 spring-security-config.jar

包含安全命令空间的解析代码。如果你使用Spring Security XML命令空间进行配置你需要包含这个模块。主包名 `org.springframework.security.config`. 没有类需要被应用程序直接使用。

LDAP spring-security-ldap.jar

LDAP 验证和配置代码. 如果你需要使用LDAP验证和管理LDAP用户实体，你需要这个模块。主包名 `org.springframework.security.ldap`.

ACL 访问控制列表- spring-security-acl.jar

ACL专门的领域对象的实现. 用来在你的应用程序中应用安全到特定的领域对象实例。主包名为 `org.springframework.security.acls`.

CAS spring-security-cas.jar

Spring Security的CAS 客户端集成. 如果你想用CAS的SSO服务器使用Spring Security 网页验证 需要该模块。顶部的包是 `org.springframework.security.cas`.

OpenID spring-security-openid.jar

OpenID 网页验证支持. 使用外部的OpenID 服务器验证用户. `org.springframework.security.openid`. 需要 OpenID4Java.

签出源代码

因为Spring Security是一个开源项目，我们强烈建议您使用Git签出源代码。这可以让你完全访问所有示例应用程序，你可以轻松地构建最先进的最新版本的项目。 有了项目的源代码对debug有非常大的帮助. 异常堆栈跟踪不再是模糊的黑盒问题，你可以沿着他找出到底发生了什么. 源代码是终极文档，通常也是最简答的找出项目实际是如何工作的地方。

要获取项目源代码，请使用以下git命令：

```
git clone https://github.com/spring-projects/spring-security.git
```

这可以让你在本地机器上访问到整个项目的历史记录（包括所有版本和分支）。

Spring Security 4 新特性

Spring Security 4.0 release解决了175+的 tickets 。

功能

下面Spring Security 4.0的新功能的亮点：

- Web Socket 支持
- Test 支持
- Spring Data 集成
- CSRF Token 参数解析器
- 更多安全默认值
- 角色方法不需要包含 ROLE_ 例如 之前需要在XML配置中包含以下内容:

```
<intercept-url pattern="/**" access="hasRole('ROLE_USER')"/>
```

现在你可以选择忽略ROLE_ 前缀. 我们这样做是为了消除重复. 具体地，由于hasRole表达式已将值定义为一个角色，如果它不存在会自动添加前缀。下面的配置和前面的配置效果一样:

```
<intercept-url pattern="/**" access="hasRole('USER')"/>
```

类似的下面的配置

```
@PreAuthorize("hasRole('ROLE_USER')")
```

和下面更简洁的配置是相同的

```
@PreAuthorize("hasRole('USER')")
```

- 很多集成测试添加到示例中
- 弃用@EnableWebMvcSecurity - 通过更新最小Spring版本，我们现在默认使用@EnableWebSecurity 和MVC集成，仍然允许它被覆盖。

从 3.x 迁移到 4.x

Spring Security为了应对应用漏洞的发展，作为一个主要版本，Spring Security团队趁机做出一些非被动的变化，这集中在：

- 确保默认配置下更安全
- 尽量避免信息泄露
- 移除废弃的API

查看完成的从Spring Security 3到4 请参考以下指南：

Migrating from Spring Security 3.x to 4.x (XML Configuration)
Migrating from Spring Security 3.x to 4.x (Java Configuration)

Java配置

Java配置的支持主要在Spring框架的3.1加入。Spring Security 从3.2开始加入java配置支持，这让用户不使用任何XML用更简单方式配置Spring Security.

如果你熟悉第4章，安全命名空间配置，你应该找到它和Java的安全配置支持之间的相当多的相似之处。

> Spring Security 提供大量的示例应用，使用`-jc`参数演示使用Java 配置Spring Security

基础的网站安全 Java配置

第一步是创建我们的Java配置。这个配置在你的应用程序中创建一个springSecurityFilterChain 的Servlet的过滤器，负责所有安全（例如 保护应用程序的URL,验证提交的用户名和密码，重定向到登陆的表单等等）。你可以在下面找到大部分Java配置项的例子：

```
import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.context.annotation.*;
import org.springframework.security.config.annotation.authentication.builders.*;
import org.springframework.security.config.annotation.web.configuration.*;

@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
        auth.inMemoryAuthentication()
            .withUser("user").password("password").roles("USER");
    }
}
```

> configureGlobal 方法的名字不重要，然而，重要的是只在一个被@EnableWebSecurity, @EnableGlobalMethodSecurity或者@EnableGlobalAuthentication注解的类中配置AuthenticationManagerBuilder，否则会有不可预知的后果。

真的是没有太多的配置，但它确实很多功能。你可以在下面的找到功能摘要：

- 在你的应用程序中对每个URL进行验证
- 为你生成一个登陆表单
- 允许用户使用用户名user和密码password使用验证表单进行验证。
- 允许用户登出
- CSRF 攻击防范
- Session保护
- 安全 Header 集成
  - 对安全要求严格的HTTP传输安全
  - X-Content-Type-Options 集成
  - 缓存控制(稍后可以允许你缓存静态资源)
  - X-XSS-Protection集成
  - X-Frame-Options集成防止点击劫持
- 和以下 Servlet API 方法集成
  - HttpServletRequest#getRemoteUser()
  - HttpServletRequest.html#getUserPrincipal()
  - HttpServletRequest.html#isUserInRole(java.lang.String)
  - HttpServletRequest.html#login(java.lang.String, java.lang.String)
  - HttpServletRequest.html#logout()

AbstractSecurityWebApplicationInitializer

下一步是在war里注册 springSecurityFilterChain. 这可以通过Spring在Servlet 3.0+环境中对WebApplicationInitializer的支持进行Java配置，Spring Security提供了基本的抽象类 `AbstractSecurityWebApplicationInitializer` ,这可以确保springSecurityFilterChain被注册。我们使用 `AbstractSecurityWebApplicationInitializer` 的不同方式取决于你是已经在使用Spring框架还是只使用了Spring Security。

- Section 3.1.2, “AbstractSecurityWebApplicationInitializer 不与Spring一起使用” - 使用这个说明如果你没有使用Spring框

架

- Section 3.1.3, “AbstractSecurityWebApplicationInitializer 与Spring一起使用” - 如果你正在使用Spring框架使用这个说明

AbstractSecurityWebApplicationInitializer 不与Spring一起使用

如果你没有使用Spring MVC 或Spring , 你需要传递 `SecurityConfig` 到超类来确保配置被使用，你可以参考下面的例子：

```
import org.springframework.security.web.context.*;

public class SecurityWebApplicationInitializer
    extends AbstractSecurityWebApplicationInitializer {

    public SecurityWebApplicationInitializer() {
        super(SecurityConfig.class);
    }
}
```

`SecurityWebApplicationInitializer` 将会做下面的事情:

- 自动为你的应用程序的每个URL注册 `springSecurityFilterChain` 过滤器
- 添加一个 `ContextLoaderListener` 用来载入 `SecurityConfig` .

AbstractSecurityWebApplicationInitializer 与Spring一起使用

如果我们在应用程序的其他地方已经使用了Spring,那么我们已经有了一个 `WebApplicationInitializer` 用来载入Spring的配置。如果我们使用上面的配置将会得到一个错误。所以我们应该使用已经存在的 `ApplicationContext` 来注册Spring Security. 举个例子，如果我们使用Spring MVC我们的 `SecurityWebApplicationInitializer` 应该看起来和下面的差不多：

```
import org.springframework.security.web.context.*;

public class SecurityWebApplicationInitializer
    extends AbstractSecurityWebApplicationInitializer {

}
```

这会简单的只为你的应用程序的所有URL注册springSecurityFilterChain过滤器。然后我们需要确保这个Security配置被载入到我们已经存在的 `ApplicationInitializer` . 例如, 如果你使用Spring MVC他应该被加入到 getRootConfigClasses()

```
public class MvcWebApplicationInitializer extends
        AbstractAnnotationConfigDispatcherServletInitializer {

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return new Class[] { SecurityConfig.class };
    }

    // ... other overrides ...
}
```

HttpSecurity

到目前为止我们的 SecurityConfig 只包含了关于如何验证我们的用户的信息。Spring Security怎么知道我们想对所有的用户进行验证？Spring Security怎么知道我们需要支持基于表单的验证？原因是 `WebSecurityConfigurerAdapter` 在 `configure(HttpSecurity http)` 方法提供了一个默认的配置，看起来和下面类似：

```
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()
            .anyRequest().authenticated()
            .and()
        .formLogin()
            .and()
        .httpBasic();
}
```

上面的默认配置:

- 确保我们应用中的所有请求都需要用户被认证
- 允许用户进行基于表单的认证
- 允许用户使用HTTP基本验证进行认证

你可以看到这个配置和下面的XML命名配置相似：

```
<http>
    <intercept-url pattern="/**" access="authenticated"/>
    <form-login />
    <http-basic />
</http>
```

Java配置使用and()方法相当于XML标签的关闭，这样允许我们继续配置父类节点。如果你阅读代码他很合理，我想配置请求验证，并使用表单和HTTP基本身份验证进行登录。

然而，Java配置有不同的默认URL和参数，当你自定义用户登录页时需要牢记这一点。让我们的URL更加RESTful，另外不要那么明显的观察出我们在使用Spring Security这样帮助我们避免信息泄露。比如：

Java配置和表单登录

你可能会想知道系统提示您登录时登录表单从哪里来的，因为我们都没有提供任何的HTML或JSP文件。由于Spring Security的默认配置并没有明确设定一个登录页面的URL，Spring Security自动生成一个，基于这个功能被启用，使用默认URL处理登录的提交内容，登录后跳转的URL等等。

自动生成的登录页面可以方便应用的快速启动和运行，大多数应用程序都需要提供自己的登录页面。要做到这一点，我们可以更新我们的配置，如下所示：

```
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()
            .anyRequest().authenticated()
            .and()
        .formLogin()
            .loginPage("/login") //注1
            .permitAll();        //注2
}
```

1. 指定登陆页的路径
2. 我们必须允许所有用户访问我们的登录页 (例如未验证的用户)，这个formLogin().permitAll() 方法允许基于表单登录的所有URL的所有用户的访问。

一个我们当前配置使用的JSP实现的页面如下：

> 下面这个登陆页是我们的当前配置，如果不符合我们的要求我们可以很容易的更新我们的配置。

```
 <c:url value="/login" var="loginUrl"/>
<form action="${loginUrl}" method="post">   <!-- 1 -->
    <c:if test="${param.error != null}">     <!-- 2  -->
        <p>
            Invalid username and password.
        </p>
    </c:if>
    <c:if test="${param.logout != null}">    <!-- 3  -->
        <p>
            You have been logged out.
        </p>
    </c:if>
    <p>
        <label for="username">Username</label>
        <input type="text" id="username" name="username"/><!-- 4  -->
    </p>
    <p>
        <label for="password">Password</label>
        <input type="password" id="password" name="password"/><!-- 5  -->    </p>
    <input type="hidden"
        name="${_csrf.parameterName}"
        value="${_csrf.token}"/><!-- 6  -->
    <button type="submit" class="btn">Log in</button>
</form>
```

1. 一个POST请求到/login用来验证用户
2. 如果参数有错误，验证尝试失败
3. 如果请求参数logout存在则登出
4. 登录名参数必须被命名为username
5. 密码参数必须被命名为password
6. CSRF参数，了解更多查阅 后续“包含CSRF Token” 和 "跨站请求伪造(CSRF)"相关章节

验证请求

我们的例子中要求用户进行身份验证并且在我们应用程序的每个URL这样做。我们可以通过给`http.authorizeRequests()`添加多个子节点来指定多个定制需求到我们的URL。例如：

```
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()  //1
            .antMatchers("/resources/**", "/signup", "/about").permitAll() //2
            .antMatchers("/admin/**").hasRole("ADMIN") //3
            .antMatchers("/db/**").access("hasRole('ADMIN') and hasRole('DBA')")           4
            .anyRequest().authenticated()  //5
            .and()
        // ...
        .formLogin();
}
```

1. `http.authorizeRequests()`方法有多个子节点，每个macher按照他们的声明顺序执行。
2. 我们指定任何用户都可以访问的多个URL模式。任何用户都可以访问URL以 "/resources/",开头的URL ,以及"/signup", "/about".
3. 以"/admin/" 开头的URL只能由拥有 "ROLE*ADMIN*"角色的用户访问. 请注意我们使用*HasRole*方法，没有使用*ROLE*前缀。
4. 任何以/db/开头的URL需要用户同时具有"ROLE*ADMIN*" 和 *"ROLE_DBA"*. 和上面一样我们的*hasRole*方法也没有使用*ROLE*前缀。
5. 尚未匹配的任何URL要求用户进行身份验证

处理登出

当使用 `WebSecurityConfigurerAdapter` 时注销功能会被自动运用上，默认是通过访问 `/logout` 这个URL将用户登出，该操作执行以下动作：

- 使Session无效
- 清除所有已经配置的RememberMe认证
- 清除 `SecurityContextHolder`
- 跳转到 `/login?success`

和登录功能类似，你也有不同的选项来定制你的注销功能。

```
protected void configure(HttpSecurity http) throws Exception {
    http
        .logout()  // 1
            .logoutUrl("/my/logout")// 2
            .logoutSuccessUrl("/my/index") // 3
            .logoutSuccessHandler(logoutSuccessHandler)//4
            .invalidateHttpSession(true)  // 5
            .addLogoutHandler(logoutHandler) // 6
            .deleteCookies(cookieNamesToClear) //7
            .and()
        ...
}
```

1. 提供注销支持，使用 `WebSecurityConfigurerAdapter.` 是会自动被应用。
2. 设置触发注销操作的URL(默认是logout),如果CSRF保护被启用（默认是启用的）的话这个请求的方式被限定为POST.请查阅javaDoc相关信息。
3. 注销之后跳转的URL.默认是 `/login?logout` ,具体查看JaveDoc文档
4. 让你设置定制的 `LogoutSuccessHandler` ,如果指定了这个选项那么 `logoutSuccessUrl()` 的设置会被忽略，请查阅JavaDoc文档
5. 指定是否在注销时让HttpSession无效。默认设置为true,在内部配置 `SecurityContextLogoutHandler` 选项。请参阅JavaDoc
6. 添加一个 `logoutHandler` ,默认 `SecurityContextLogoutHandler` 会被添加为最后一个logoutHandler.
7. 允许指定在注销成功时将移除的cookie.这是一个显示的添加一个 `CookieClearingLogoutHandler` 的快捷方式。

> 注销也可以通过XML命名空间进行配置，请参阅Spring Security XML命令空间相关文档获取更多细节。

一般来说，为了定制注销功能，你可以添加 `LogoutHandler` 以及 `LogoutSuccessHandler` 的实现。对于许多常见场景，当使用流式API时，这些处理器会在幕后进行添加。

LogoutHandler

一般来说， `LogoutHandler` 的实现类可以参与到注销处理中。他们被用来执行必要的清理，因而他们不应该抛出错误，我们提供各种实现：

- PersistentTokenBasedRememberMeServices
- TokenBasedRememberMeServices
- CookieClearingLogoutHandler
- CsrfLogoutHandler
- SecurityContextLogoutHandler

请查看 "Remember-Me接口和实现"获取详情。

流式API提供了调用相应的 `LogoutHandler` 实现的快捷方式，而不用直接提供 `LogoutHandler` 的实现。例如： `deleteCookies()`

允许指定注销成功时要删除的一个或者多个cookie.这是一个添加 `CookieClearingLogoutHandler` 的快捷方式。

LogoutSuccessHandler

`LogoutSuccessHandler` 被 `LogoutFiler` 在成功注销后调用，用来进行重定向或者转发相应的目的地。注意这个接口与LogoutHandler几乎一样，但是可以抛出异常。

下面是 提供的一些实现：

- SimpleUrlLogoutSuccessHandler
- HttpStatusReturningLogoutSuccessHandler

和前面提到的一样，你不需要直接指定 `SimpleUrlLogoutSuccessHandler` .而使用流式API通过设置 `logoutSuccessUrl()` 快捷的进行设置。这样会隐式的设置 `SimpleUrlLogoutSuccessHandler` . 注销成功后将重定向到设置的URL地址。默认的地址是 `/login?logout` .

在REST API场景中 `HttpStatusReturningLogoutSuccessHandler` 会进行一些有趣的改变。 `LogoutSuccessHandler` 允许你设置一个返回给客户端的HTTP状态码（默认返回200）来替换重定向到URL这个动作.

进一步的注销相关的参考(TODO)

- 处理注销
- 测试注销
- HttpServletRequest.logout()
- 章节 “Remember-Me 接口和实现”
- 注销的CSRF说明
- 单点注销(CAS协议)
- 注销的XML命名空间章节

验证

到现在为止我们只看了一下基本的验证配置，让给我们看看一些稍微高级点的身份验证配置选项。

内存中的身份验证

我们已经看到了一个单用户配置到内存验证的示例，下面是配置多个用户的例子：

```
@Autowired
public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
    auth
        .inMemoryAuthentication()
            .withUser("user").password("password").roles("USER").and()
            .withUser("admin").password("password").roles("USER", "ADMIN");
}
```

JDBC 验证

你可以找到一些更新来支持基于JDBC的验证。下面的例子假设你已经在应用程序中定义好了 `DateSource` , jdbc-jc示例提供了一个完整的基于JDBC的验证。

```
@Autowired
private DataSource dataSource;

@Autowired
public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
    auth
        .jdbcAuthentication()
            .dataSource(dataSource)
            .withDefaultSchema()
            .withUser("user").password("password").roles("USER").and()
            .withUser("admin").password("password").roles("USER", "ADMIN");
}
```

LDAP 验证

你可以找到一些更新来支持基于LDAP的身份验证，ldap-jc提供一个完成的使用基于LDAP的身份验证的示例。

```
@Autowired
private DataSource dataSource;

@Autowired
public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
    auth
        .ldapAuthentication()
            .userDnPatterns("uid={0},ou=people")
            .groupSearchBase("ou=groups");
}
```

上面的例子中使用以下LDIF和嵌入式Apache DS LDAP实例。

users.ldif.

```
dn: ou=groups,dc=springframework,dc=org
objectclass: top
objectclass: organizationalUnit
```

```
ou: groups

dn: ou=people,dc=springframework,dc=org
objectclass: top
objectclass: organizationalUnit
ou: people

dn: uid=admin,ou=people,dc=springframework,dc=org
objectclass: top
objectclass: person
objectclass: organizationalPerson
objectclass: inetOrgPerson
cn: Rod Johnson
sn: Johnson
uid: admin
userPassword: password

dn: uid=user,ou=people,dc=springframework,dc=org
objectclass: top
objectclass: person
objectclass: organizationalPerson
objectclass: inetOrgPerson
cn: Dianne Emu
sn: Emu
uid: user
userPassword: password

dn: cn=user,ou=groups,dc=springframework,dc=org
objectclass: top
objectclass: groupOfNames
cn: user
uniqueMember: uid=admin,ou=people,dc=springframework,dc=org
uniqueMember: uid=user,ou=people,dc=springframework,dc=org

dn: cn=admin,ou=groups,dc=springframework,dc=org
objectclass: top
objectclass: groupOfNames
cn: admin
uniqueMember: uid=admin,ou=people,dc=springframework,dc=org
```

多个HttpSecurity

我们可以配置多个HttpSecurity实例，就像我们可以有多个块抑郁。关键在于对 `WebSecurityConfigurationAdapter` 进行多次扩展。例如下面是一个对 `/api/` 开头的URL进行的不同的设置。

```
@EnableWebSecurity
public class MultiHttpSecurityConfig {
    @Autowired
    public void configureGlobal(AuthenticationManagerBuilder auth) { //1
        auth
            .inMemoryAuthentication()
                .withUser("user").password("password").roles("USER").and()
                .withUser("admin").password("password").roles("USER", "ADMIN");
    }

    @Configuration
    @Order(1)  // 2
    public static class ApiWebSecurityConfigurationAdapter extends WebSecurityConfigurerAdapter {
        protected void configure(HttpSecurity http) throws Exception {
            http
                .antMatcher("/api/**")  // 3
                .authorizeRequests()
                    .anyRequest().hasRole("ADMIN")
                    .and()
                .httpBasic();
        }
    }

    @Configuration // 4
    public static class FormLoginWebSecurityConfigurerAdapter extends WebSecurityConfigurerAdapter {

        @Override
        protected void configure(HttpSecurity http) throws Exception {
            http
                .authorizeRequests()
                    .anyRequest().authenticated()
                    .and()
                .formLogin();
        }
    }
}
```

1. 配置正常的验证
2. 创建一个 `WebSecurityConfigurerAdapter` ，包含一个 `@Order` 注解，用来指定哪一个 `WebSecurityConfigurerAdapter` 更优先。
3. `http.antMatcher` 指出，这个 `HttpSecurity` 只应用到以 `/api/` 开头的URL上。
4. 创建另外一个 `WebSecurityConfigurerAdapter` 实例。用于不以 `/api/` 开头的URL，这个配置的顺序在 `ApiWebSecurityConfigurationAdapter` 之后，因为他没有指定 `@Order` 值 (没有指定 `@Order` 默认会被放到最后).

方法安全

从2.0开始Spring Security对服务层的方法的安全有了实质性的改善。他提供对JSR-250的注解安全支持象框架原生的@Secured注解一样好。从3.0开始你也可以使用新的基于表达式的注解。你可以应用安全到单独的bean,使用拦截方法元素去装饰Bean声明，或者你可以在整个服务层使用AspectJ风格的切入点保护多个bean。

EnableGlobalMethodSecurity

我们可以在任何使用 `@Configuration` 的实例上，使用 `@EnableGlobalMethodSecurity` 注解来启用基于注解的安全性。例如下面会启用Spring 的 @Secured注解.

```
@EnableGlobalMethodSecurity(securedEnabled = true)
public class MethodSecurityConfig {
// ...
}
```

添加一个注解到一个方法（或者一个类或者接机）会限制对相应方法的访问。Spring Security的原生注解支持定义了一套用于该方法的属性。这些将被传递到 `AccessDecisionManager` 来做实际的决定：

```
public interface BankService {

@Secured("IS_AUTHENTICATED_ANONYMOUSLY")
public Account readAccount(Long id);

@Secured("IS_AUTHENTICATED_ANONYMOUSLY")
public Account[] findAccounts();

@Secured("ROLE_TELLER")
public Account post(Account account, double amount);
}
```

使用如下代码启用JSR-250注解的支持

```
@EnableGlobalMethodSecurity(jsr250Enabled = true)
public class MethodSecurityConfig {
// ...
}
```

这些都是基于标准的，并允许应用简单的基于角色的约束，但是没有Spring Security的本地注解的能力。要使用新的基于表达式的语法，你可以使用：

```
@EnableGlobalMethodSecurity(prePostEnabled = true)
public class MethodSecurityConfig {
// ...
}
```

和响应的Java代码如下：

```
public interface BankService {

@PreAuthorize("isAnonymous()")
public Account readAccount(Long id);

@PreAuthorize("isAnonymous()")
```

```
public Account[] findAccounts();

@PreAuthorize("hasAuthority('ROLE_TELLER')")
public Account post(Account account, double amount);
}
```

GlobalMethodSecurityConfiguration

有时候你可能需要执行一些比`@ EnableGlobalMethodSecurity`注解允许的更复杂的操作。对于这些情况，你可以扩展GlobalMethodSecurityConfiguration确保`@EnableGlobalMethodSecurity`注解出现在你的子类。例如：如果你想提供一个定制的`MethodSecurityExpressionHander`，你可以使用下面的配置：

```
@EnableGlobalMethodSecurity(prePostEnabled = true)
public class MethodSecurityConfig extends GlobalMethodSecurityConfiguration {
    @Override
    protected MethodSecurityExpressionHandler createExpressionHandler() {
        // ... create and return custom MethodSecurityExpressionHandler ...
        return expressionHander;
    }
}
```

关于可以被重载的方法的更多信息，请参考GlobalMethodSecurityConfiguration的java文档。

已配置对象的后续处理

Spring Security的Java配置没有公开每个配置对象的每一个属性，这简化了广大用户的配置。毕竟如果要配置每一个属性，用户可以使用标准的Bean配置。

虽然有一些很好的理由不直接暴露所有属性，用户可能任然需要更多高级配置，为了解决这个Spring Security引入了`ObjectPostProcessor`概念，用来修改或替换Java配置的对象实例。例如：如果你想在`FilterSecurityInterceptor`里配置`filterSecurityPublishAuthorizationSuccess`属性，你可以像下面一样：

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()
            .anyRequest().authenticated()
            .withObjectPostProcessor(new ObjectPostProcessor<FilterSecurityInterceptor>() {
                public <O extends FilterSecurityInterceptor> O postProcess(
                        O fsi) {
                    fsi.setPublishAuthorizationSuccess(true);
                    return fsi;
                }
            });
}
```

安全命名空间配置

简介

命名空间配置在Spring框架的2.0版本就可以使用了，他允许你通过额外的XML架构元素补充传统的Spring bean应用程序上下文。你可以从Spring的参考文档找到更多信息。命名空间元素可以简单的允许配置单个bean,或者更强大的，定义一个可选的配置语法，这样更贴近问题域并且对用户隐藏背后的复杂性。一个简单的元素可以隐藏多个bean 和添加到应用程序上下文的多个处理步骤。例如：从安全命令空间添加后面的元素到应用程序上下文将开始一个LDAP服务到应用程序内用于测试：

```
<security:ldap-server />
```

这比配置一个Apache目录服务器bean简单得多。最常见的替代配置需求是 `ldap-server` 元素的属性支持，用户不用担心他们要创建的bean的属性名称（注：你可以从Chapter 26, LDAP 身份验证找到如何使用 `ldap-server` 元素的更多信息 ）。当编辑应用的Context文件时良好的XML编辑器应该可以提供可用的属性和元素的信息。我们建议你尝试使用SpringSource Tool Suite，它具有与标准Spring 命名空间工作的特别的功能。

为了在你的应用程序上下文中使用安全命名空间，你需要将 `spring-security-config` jar包含到你的classpath中。然后在你的上下文文件中加入以下的结构声明：

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:security="http://www.springframework.org/schema/security"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
        http://www.springframework.org/schema/security
        http://www.springframework.org/schema/security/spring-security.xsd">
    ...
</beans>
```

在很多示例（包括示例应用程序）中你将会看到，我们经常使用 `security` 作为默认的命名空间而不是使用 `beans`,这样我们可以在所有安全命名空间中忽略前缀，使得内容更加容易阅读。如果你的的应用程序上下文被分割成单独的文件，大部分的安全配置被放到一个文件中，你可能也想这样做。你的安全应用上下文文件应该像下面的一样：

```
<beans:beans xmlns="http://www.springframework.org/schema/security"
xmlns:beans="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
        http://www.springframework.org/schema/security
        http://www.springframework.org/schema/security/spring-security.xsd">
    ...
</beans:beans>
```

我们假设在这一章节我们都使用这种语法。

命名空间的设计

命名空间是设计用来扑捉框架最常见用途和提供一个简化和简介的语法用来在应用程序中打开他们。设计是基于框架中的大规模依赖，并且可以划分为以下几个方面：

- Web/HTTP 安全- 最复杂的部分，设置过滤器和应用框架验证机制的相关服务bean,渲染登录和错误页面等等。
- 业务对象（方法）安全 - 业务层安全选项.
- AuthenticationManager - 处理来自框架其他部分的认证请求
- AccessDecisionManager - 为网页和方法安全提供访问决策，会注册一个默认的但是你可以使用一个定制的来取代他，使用一般的Spring bean语法即可定义.
- AuthenticationProviders - 认证管理器认证用户的机制，命名空间提供多种标准选项的支持，同时使用传统语法添加自定义bean的方法。
- UserDetailsService - 和AuthenticationProviders密切相关，但往往也被其他bean需要。

我们将在后续章节查看怎么配置他们。

开始使用安全命名空间配置

在这一章，我们将看看怎么创建一个命名空间配置来使用框架的主要功能。让我们假设你想要快速的使用命名空间配置添加验证支持和访问控制和一些测试的登录到一个已经存在的网站应用程序。然后我们看看如何将验证换刀数据库和其他的安全仓库。在后续章节我们将介绍更多高级的命名空间配置选项。

web.xml 配置

你需要做的第一件事情是添加下面的过滤器定义到你的 `web.xml` 文件:

```
<filter>
<filter-name>springSecurityFilterChain</filter-name>
<filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
</filter>

<filter-mapping>
<filter-name>springSecurityFilterChain</filter-name>
<url-pattern>/*</url-pattern>
</filter-mapping>
```

这提供了一个钩子到Spring Security的网页基础设施。 `DelegatingFilterProxy` 是一个委托了在应用的Context中定义为bean的过滤器实现的一个Spring框架类，在这种情况下，bean的名字是 `springSecurityFilterChain` ,这是网络安全命名空间创建的一个基础类。注意你不能应该自己使用这个bean名字。一旦你添加这个到你的 `web.xml` ,你就可以开始编辑你的应用程序上下文文件,网站安全服务被配置到 `http` 元素。

最小 `<http>` 配置

开始开启网页安全你只需要：

```
<http>
<intercept-url pattern="/**" access="hasRole('USER')" />
<form-login />
<logout />
</http>
```

这表示，我们想要对我们应用程序中的所有URL进行安全性检测。需要角色"ROLE*USER*'访问他们，我们使用一个包含*username*和*password*的表单登录到应用程序，我们希望有一个注销的*URL*，让我们可以登出应用程序。 `<http>` 元素是所有网站相关的命名空间功能的父元素。 `<intercept-url>` 元素定义了一个规则使用*ant*语法去匹配传入的请求的*URL*.（在网站应用基础设施章节，查看 请求匹配和 *HttpFirewall* 来了解匹配是如何执行的）,你也可以换成正则表达式语法(查看附录了解更多)。这个 `access` 属性定义了这个匹配的规则需要的访问需求。默认是一个典型的用逗号分隔的角色列表，有其中一个符合则允许执行这个请求。前缀```*ROLE*```是一个标记，指示一个和用户的授权进行简单的比较。换句话说，一个简单的基于角色的检查会被应用。Spring Security的访问控制不仅限与简单角色（因此使用前缀来区分不同类型的安全属性）。后面我们会看到解释如何变化的注脚：[逗号分隔值的解释在访问属性取决于所使用的–1—实施。在Spring Security3.0，属性还可以填充一个–2—。（//_-这是啥。。郁闷）

> 你可以使用多个 `<intercept-url>` 元素来为不同的URL定于不同的访问需求,他是他们将按照顺序计算，第一个匹配的将会被使用，所以你必须将最特别的匹配放到最上面，你也可以添加一个 `method` 属性用来限制HTTP方法(GET, POST, PUT 等等.).

为了添加一些用户，你可以直接在命名空间直接定义一组测试数据。

```
<authentication-manager>
<authentication-provider>
    <user-service>
    <user name="jimi" password="jimispassword" authorities="ROLE_USER, ROLE_ADMIN" />
    <user name="bob" password="bobspassword" authorities="ROLE_USER" />
    </user-service>
</authentication-provider>
</authentication-manager>
```

如果你熟悉框架的预命名空间版本，你很可能已经猜到这里是怎么回事了。`<http>` 元素负责创建 `FilterChainProxy` 和它使用的过滤器bean.像不正确过滤排序常见的问题是不再是一个问题，因为过滤器的位置都是预定义的。

`<authentication-provider>` 元素创建一个 `DaoAuthenticationProvider` bean并且 `<user-service>` 元素创建了一个 `InMemoryDaoImpl` 。所有的严重供应商元素必须是 `<authentication-manager>` 元素的子元素。他创建一个 `ProviderManager` 并注册为验证供应商。你可以在命名空间目录找到更多信息。如果你想了解框架的重要的一些类了解他们如何使用特别是后面你想定制一些事情，你值得去查看一下。

上面的配置定义了2个用户，他们的密码和在这个应用程序里的角色（这将用于访问控制）。也可以使用用户服务从标准的属性文件载入用户信息。查看内存认证的部分文件格式的更多细节。使用 `<authentication-provider>` 元素意思是这些用户信息将被认证管理器使用到请求验证。你可以设置多个 `<authentication-provider>` 元素来定义不同的验证源每一个都会被依次访问到。

此时你应该可以开始你的应用程序，你将被要求登录。尝试一下或者尝试使用附带的项目教程示例。

表单和基本登录选项

你可能会想当系统提示你登录这些登录表单哪里来的，因为我们都没用提供任何的HTML和JSP文件。事实上我们并没有明确的设定一个登陆页的URL,Spring Security自动生成了一个，基于处理登录的URL标准值。以及登录后跳转的URL.然后命名空间提供了大量的支持，让你可以自定义这些选项，例如如果你想自己设计登录页面，你可以使用：

```
<http>
<intercept-url pattern="/login.jsp*" access="IS_AUTHENTICATED_ANONYMOUSLY"/>
<intercept-url pattern="/**" access="ROLE_USER" />
<form-login login-page='/login.jsp'/>
</http>
```

另外请注意，我们添加了一个额外的intercept-url 元素，来指定登陆页的请求可以被任何匿名用户访问，(`AuthenticatedVoter` 类可以提供更多 `IS_AUTHENTICATED_ANONYMOUSLY` 被处理的细节)。 否则请求会被 `/**` 规则匹配这将是不可能访问到登录页面。这是一个常见的配置错误，将导致应用无限循环。Spring Security会产生一条经过到日志中如果你的登录页被保护了。它也可以设置让所有请求完全匹配特定模式而绕过安全过滤器，通过定义像这样单独http元素的规则：

```
<http pattern="/css/**" security="none"/>
<http pattern="/login.jsp*" security="none"/>

<http use-expressions="false">
<intercept-url pattern="/**" access="ROLE_USER" />
<form-login login-page='/login.jsp'/>
</http>
```

从Spring Security 3.1开始允许使用多个 `http` 元素来为不同的请求规则分别定义安全过滤器链配置。如果 `http` 元素的 `pattern` 属性未设置则匹配所有请求。创建不进行安全检查的规则是这个语法的一个简单的例子，这个规则会被映射到一个空的过滤链。我们将会在后面的 安全过滤器链章节找到更多细节。

> 多个 `<http>` 元素的使用是一个非常重要的功能， 例如它允许命名空间在一个应用程序中同时支持有状态和无状态的相同URL.之前的语法，使用 `filters="none"` 属性到一个 `intercept-url` 元素与这一变化想抵触，在3.1将不再进行支持。

重要的的一点需要认识到，这些不安全的请求将完全无视任何Spring Security的web相关的配置或附加属性，例如通道的限定，所以在请求期间你将无法访问当前用户信息，或调用需要进行安全验证的方法。如果你想让安全过滤器链被应用另外也可以使用 `access=IS_AUTHENTICATED_ANONYMOUSLY` 。

如果你想使用基本验证来替换掉基于表单的登录，你可以将配置修改如下：

```
<http use-expressions="false">
<intercept-url pattern="/**" access="ROLE_USER" />
<http-basic />
</http>
```

当用户访问被保护的资源时，基本身份验证将优先弹出提示框来进行登录。在这种配置下如果你想使用基于表单的登录，他任然有效。例如，将登录嵌入另外一个网页。

设置默认的登录后地址

如果不是由试图访问受保护的资源促成的登录，默认目标URL就会起作用。这是一个当用户成功登录后会被带去的URL默认是 `/` ，你也可以通过配置 `always-use-default-target` 为 `true` 让用户一直跳转到这个地址而无论用户是按需还是明确的选择登录。这在你的应用程序需要用户一直从主页开始时很有用。例如：

```
<http pattern="/login.htm*" security="none"/>
<http use-expressions="false">
<intercept-url pattern='/**' access='ROLE_USER' />
<form-login login-page='/login.htm' default-target-url='/home.htm'
        always-use-default-target='true' />
</http>
```

为了对跳转目标进行更多控制，你可以使用 `authentication-success-handler-ref` 来替代 `default-target-url` .这个bean的引用应该是 `AuthenticationSuccessHandler` 的一个实例。你可以在核心过滤器章节和命名空间附录找到如何定制认证失败的流程的更多信息。

处理登出

`logout` 元素，增加了一个登出导航到一个特定URL的支持，默认的登出URL是 `/logout` 。但是你可以使用 `logout-url` 属性指定为其他的URL .更多的属性你可以从附录找到。

使用其他的验证供应商

在实践中，你将需要比将一些名字添加到应用程序上下文文件中更具扩展性的数据源。你有可能想把用户信息存放在诸如数据库或者LDAP服务器中。LDAP命名空间的配置在香港的章节， 这里我们就不细说。如果你的应用程序上下文中有一个Spring Security的 UserDetailsService 的实现叫做 "myUserDetailsService",你可以这样使用：

```
<authentication-manager>
    <authentication-provider user-service-ref='myUserDetailsService'/>
</authentication-manager>
```

其中“securityDataSource”是在应用程序上下文一个DataSource的 bean的名称，它指向了包含着Spring Security用户信息的表。或者，你可以在这个配置Spring Security的 `JdbcDaoImpl` 的bean和使用 `user-service-ref` 属性：

```
<authentication-manager>
<authentication-provider user-service-ref='myUserDetailsService'/>
```

```
</authentication-manager>

<beans:bean id="myUserDetailsService"
    class="org.springframework.security.core.userdetails.jdbc.JdbcDaoImpl">
<beans:property name="dataSource" ref="dataSource"/>
</beans:bean>
```

你也可以像下面一样使用标准的 `AuthenticationProvider`

```
<authentication-manager>
    <authentication-provider ref='myAuthenticationProvider'/>
</authentication-manager>
```

`myAuthenticationProvider` 是应用程序上下文的一个bean的名字，他实现了 `AuthenticationProvider` 接口。你可以设置多个验证供应商，这样这些供应商就会被按照定于的顺序被查询。看看章节 "验证管理器和命名空间" 了解 `AuthenticationManager` 在命名空间中怎么被配置。

添加密码编码器

密码应该一直使用一个尽可能安全的哈希算法进行编码（非标准的算法，例如SHA或者MD5）.这在 `<password-encoder>` 中进行支持，使用 `bcrypt` 编码密码，原始的供应商配置应该类似这样：

```
<beans:bean name="bcryptEncoder"
    class="org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder"/>

<authentication-manager>
<authentication-provider>
    <password-encoder ref="bcryptEncoder"/>
    <user-service>
    <user name="jimi" password="d7e6351eaa13189a5a3641bab846c8e8c69ba39f"
            authorities="ROLE_USER, ROLE_ADMIN" />
    <user name="bob" password="4e7421b1b8765d8f9406d87e7cc6aa784c4ab97f"
            authorities="ROLE_USER" />
    </user-service>
</authentication-provider>
</authentication-manager>
```

`Bcrypt` 在大部分时候是一个好的选择。除非你有一个旧系统强迫你使用一个不同的算法。如果你正在使用一个简单的哈希算法或者更糟糕存储了明文密码，你应该考虑迁移到 `bcrypt` 这个更安全的选项。

高级Web功能

Remember-Me 验证

查看独立的 Remember-Me 章节来查看该功能的配置。

添加 HTTP/HTTPS 通道安全

如果您的应用程序同时支持HTTP和HTTPS，你要求特定的URL只能使用HTTPS，这时可以直接使用 `<intercept-url>` 的 `requires-channel` 属性：

```
<http>
<intercept-url pattern="/secure/**" access="ROLE_USER" requires-channel="https"/>
<intercept-url pattern="/**" access="ROLE_USER" requires-channel="any"/>
...
</http>
```

使用上面的配置，如果用户访问到匹配 `/secure/**` 规则的URL，如果使用了HTTP，他们将首先被重定向到HTTPS的URL.

> 如果要了解通道处理器怎么实现的，请查看 `ChannelProcessingFilter` 和相关类的API文档.

该选项可选值为"http", "https" 或者 "any"，使用"any"意思是使用HTTP或者HTTPS均可。 如果应用程序使用HTTP和/或HTTPS非标准的端口，您可以指定如下端口映射的列表：:

```
<http>
...
<port-mappings>
    <port-mapping http="9080" https="9443"/>
</port-mappings>
</http>
```

需要注意的是，为了实现真正的安全，应用程序不应该使用HTTP或在HTTP和HTTPS之间切换。它应该在由HTTPS开始（让用户进入一个HTTPS URL），并一直使用安全连接，以避免被进行中间人攻击的可能。

Session管理

检测超时

你可以配置Spring Security检测无效的Session ID提交并且将用户重定向到一个适当的URL,这通过 `session-management` 元素来达到：

```
<http>
...
<session-management invalid-session-url="/invalidSession.htm" />
</http>
```

请注意，如果你使用这个机制来检测会话超时，如果用户注销，然后重新登录，但不关闭浏览器它可能会错误地抛出一个错误。这是因为当你让Session失效时，Cookie没有被清理干净，就算用户已注销还是会重新提交session的cookie而不会清除。您可以在注销时明确的删除JSESSIONID的cookie，例如，通过在注销处理程序使用以下语法：

```
<http>
<logout delete-cookies="JSESSIONID" />
</http>
```

不幸的是这个不保证在所有servlet容器都会正常工作，所以你需要在你的环境下进行测试。

> 如果你的应用程序运行在代理后面，你还可以配置代理服务器去删除session的cookie..例如，使用Apache httpd的mod_headers模块，下面的指令可以通过在注销请求的响应头删除JSESSIONID的cookie.(假设应用程序部署在/tutorial 路径下):

```
<LocationMatch "/tutorial/logout">
Header always set Set-Cookie "JSESSIONID=;Path=/tutorial;Expires=Thu, 01 Jan 1970  00:00:00 GMT"
</LocationMatch>
```

并发Session控制

如果你想限制单个用户访问你的应用程序的能力。Spring Security通过后面简单的配置马上启用。首先你需要添加后面的监听器到你的 `web.xml` 文件。让Spring Security获得session的生存事件：

```
<listener>
<listener-class>
    org.springframework.security.web.session.HttpSessionEventPublisher
</listener-class>
</listener>
```

然后添加后面的行刀你的应用程序上下文：

```
<http>
...
<session-management>
    <concurrency-control max-sessions="1" />
</session-management>
</http>
```

这将防止用户登录多次 - 第二登录将导致第一次变成无效。通常我们更想防止第二次登录，在这种情况下，这种情况你可以使用：

```
<http>
...
<session-management>
    <concurrency-control max-sessions="1" error-if-maximum-exceeded="true" />
</session-management>
</http>
```

第二次登录将被拒绝，如果基于表单的验证被启用这个用户将会被发送到 `authentication-failure-url` . 如果第二次登录是通过其他非交互的机制，比如 "记住我" 功能，进行登录的。那么一个，"unauthorized" (401) 错误将会被发送给客户端。如果你想替换成一个错误页面，你可以为 `session-management` 添加一个 `session-authentication-error-url` 属性。

如果您正在使用基于表单的登录定制验证过滤器，那么你必须明确地配置同步会话控制的支持。更多的细节可以在会话管理章节中找到。

Session完成(Session Fixation)攻击保护

Session完成攻击是一个潜在的威胁。攻击者访问网站生成一个Session，然后诱使其他用户用同一个会话登录（例如：通过发送包含会话标示符作为一个参数链接）。Spring Security通过在登录时创建新的Session或者修改Session ID来应对这种情况。如果你不需要这个保护或者与一些其他需求冲突你可以通过 `<session-management>` 的 `session-fixation-protection` 属性来控制这个行为。他有4个选项：

- none - 什么都不做. 原来的会话将会被保留
- newSession - 建立一个新的干净的Session,不会复制已经存在的Session数据，（Spring Security相关的属性将会被复制）
- migrateSession - 创建一个新的Session并且拷贝所以已经存在的Session属性到新的Session,这是Servlet 3.0 及之前的容器的默认设置。
- changeSessionId - 不创建新的Session，使用Servlet容器提供的Session完成攻击保护 (`HttpServletRequest#changeSessionId()`). 这个选项只有在Servlet 3.1(Java EE7)和更新的容器下可用。在旧的容器设置这个选项会产生一个异常。在Servlet3.1和更新的容器这默认该选项。

当会话完成保护发生时，它会导致SessionFixationProtectionEvent发布到应用程序上下文,如果使用changeSessionId，这种保护也将导致任何javax.servlet.http.HttpSessionIdListener被通知，所以如果你的代码侦听这两个事件要特别小心。查看Session管理章节查看更多信息。

OpenID 支持

通过一个简单的改变，命名空间可用支持用OpenID替换或者添加到普通的基于表单的登录：

```
<http>
<intercept-url pattern="/**" access="ROLE_USER" />
<openid-login />
</http>
```

然后，您应该将自己注册为一个OpenID提供商（如myopenid.com），并添加用户信息到你的基于内存的 `<user-service>`：

```
<user name="http://jimi.hendrix.myopenid.com/" authorities="ROLE_USER" />
```

您应该能够使用myopenid.com网站登录来进行验证。另外，也可以通过设置 `openid-login` 元素的 `user-service-ref` 属性来指定一个 `UserDetailsService` bean来使用OpenID.查看前面的验证供应商章节了解更多信息。请注意，我们省略从上述用户配置中的密码属性，由于该组的用户数据只被用于加载当前的用户。内部会产生一个随机密码，防止你意外的将这个用户数据用作验证源到你的配置中的其他地方。

属性交换

OpenID的属性交换支持，作为一个例子，下面的例子尝试接受从OpenId供应商接收邮件和全名，用于应用程序中：

```
<openid-login>
<attribute-exchange>
    <openid-attribute name="email" type="http://axschema.org/contact/email" required="true"/>
    <openid-attribute name="name" type="http://axschema.org/namePerson"/>
</attribute-exchange>
</openid-login>
```

每个OpenID属性的 "type" 是一个 URI, 有一种特定的模式来确定，这个例子中是 `http://axschema.org/`. 如果属性必须在成功认证后被接收，可以设置 `required` 属性.确切的模式和属性的支持将取决于您的OpenID提供商。该属性值返回作为认证过程的一部分，随后可以使用下面的代码访问：

```
OpenIDAuthenticationToken token =
    (OpenIDAuthenticationToken)SecurityContextHolder.getContext().getAuthentication();
List<OpenIDAttribute> attributes = token.getAttributes();
```

`OpenIDAttribute` 包含了属性类型和接收到的值(在有多个属性值的情况下包含多个值)，通过查看Spring Security核心组件的技术预览章节我们可以了解更多的 `SecurityContextHolder` 类的使用方法。如果你希望使用多个身份提供者，多重属性交换配置也被支持。你可以提供多个 `attribute-exchange` 元素。在每个上面使用 `identifier-matcher` 属性。他包含一个正则表达式，会匹配由用户提供的的OpenID标识符。查看代码库的OpenID示例应用的一个示例配置。对Google ,Yahoo和MyOpenID提供了不同的属性列表。

响应头

查看如何定制头元素的更多信息请查看 安全HTTP响应头章节.

添加自己的过滤器

如果你以前使用过Spring Security，你就会知道，这个框架维护一个过滤器链，以便应用它的服务。你可能想要添加自己的过滤器到过滤器堆栈的特定位置，或者使用一个Spring Security还没有一个命名空间配置的选项的过滤器(比如CAS).或者你想使用一个标准命名空间过滤器的定制化版本，比如 `UsernamePasswordAuthenticationFilter` 他是由 `<form-login>` 元素显式的使用Bean来获取一些额外的高级配置选项。在过滤器链不直接暴露的情况下，你怎么使用命名空间配置做到这一点？

使用命名空间时过滤器的顺序始终严格执行。当创建应用程序上下文，过滤器Bean被命名空间处理代码进行排序，标准的Spring Security过滤器都具有的命名空间和一个众所周知的位置的别名。

> 在以前的版本中，排序发生在过滤器实例创建之后，在应用程序上下文后处理中。语法在3.0有一些轻微的改变，这会影响到解析 `<http>` 元素时，你自己的过滤器如何被添加到整个过滤器列表中

过滤器，别名和创建的命名空间元素/属性在下表列出，按照过滤器在链中的出现顺序列出：

Table 4.1. 标准过滤器别名和顺序

Alias	Filter Class	Namespace Element or Attribute
CHANNEL_FILTER	`ChannelProcessingFilter`	`http/intercept-url@requires-channel`
SECURITY_CONTEXT_FILTER	`SecurityContextPersistenceFilter`	`http`
CONCURRENT_SESSION_FILTER	`ConcurrentSessionFilter`	`session-management/concurrency-control`
HEADERS_FILTER	`HeaderWriterFilter`	`http/headers`
CSRF_FILTER	`CsrfFilter`	`http/csrf`
LOGOUT_FILTER	`LogoutFilter`	`http/logout`
X509_FILTER	`X509AuthenticationFilter`	`http/x509`
PRE_AUTH_FILTER	`AbstractPreAuthenticatedProcessingFilter` Subclasses	N/A
CAS_FILTER	`CasAuthenticationFilter`	N/A
FORM_LOGIN_FILTER	`UsernamePasswordAuthenticationFilter`	`http/form-login`
BASIC_AUTH_FILTER	`BasicAuthenticationFilter`	`http/http-basic`
SERVLET_API_SUPPORT_FILTER	`SecurityContextHolderAwareRequestFilter`	`http/@servlet-api-provision`

JAAS_API_SUPPORT_FILTER	`JaasApiIntegrationFilter`	`http/@jaas-api-provision`
REMEMBER_ME_FILTER	`RememberMeAuthenticationFilter`	`http/remember-me`
ANONYMOUS_FILTER	`AnonymousAuthenticationFilter`	`http/anonymous`
SESSION_MANAGEMENT_FILTER	`SessionManagementFilter`	`session-management`
EXCEPTION_TRANSLATION_FILTER	`ExceptionTranslationFilter`	`http`
FILTER_SECURITY_INTERCEPTOR	`FilterSecurityInterceptor`	`http`
SWITCH_USER_FILTER	`SwitchUserFilter`	N/A

您可以添加自己的过滤器到列表中，使用自定义过滤元件和这些名字来指定你的过滤器应该出现在的位置之一：

```
<http>
<custom-filter position="FORM_LOGIN_FILTER" ref="myFilter" />
</http>

<beans:bean id="myFilter" class="com.mycompany.MySpecialAuthenticationFilter"/>
```

你也可以使用 `after` 和 `before` 属性来让你的过滤器插入到列表中的其他过滤器的前面和后面。 `FIRST` 和 `LAST` 可以用在 `position` 属性来设置你希望将你的过滤器插入到整个列表的前面或者后面。

> 如果你插入的定制的过滤器可能会占用由命名空间创建的标准过滤器相同的位置，很重要的一点是不要错误的包含了命名空间的版本。移除所有你想替换的功能的过滤器元素。 注意，你不能一出有 `<http>` 元素本省创建的过滤器， `SecurityContextPersistenceFilter` , `ExceptionTranslationFilter` , `FilterSecurityInterceptor` ,一些其他的过滤器是默认被添加的，但是你可以禁止他们。除非你禁用Session完成攻击保护，一个 `AnonymousAuthenticationFilter` 会默认被添加，一个 `SessionManagementFilter` 也会被添加到过滤器链。

如果你替换的命名空间过滤器需要一个验证入口点（例如：在认证过程是通过尝试触发未认证用户访问以受保护资源）你也需要添加一个定制的入口点Bean.

创建定制的 AuthenticationEntryPoint

如果你没有使用表单登录，OpenID或基本验证，你可能想想我们之前看到的一样，使用传统的bean语法，定义一个验证过滤器和入口点链接到命名空间。相应的 `AuthenticationEntryPoint` 可以使用 `<http>` 元素的 `entry-point-ref` 属性进行设置。

CAS示例应用是一个很好的示例，展示命名空间的定制bean的使用。如果你不熟悉认证的入口点，可以在技术预览章节看到相关讨论。

方法安全

从2.0版本开始Spring Security 改进了对服务层方法的安全支持，它提供了对JSR-250注解安全支持以及框架的原生 `@Secured` 注解的支持。从3.0开始你也可以使用新的基于表达式的注解。你可以将安全应用到单个bean.使用 `intercept-methods` 元素装饰Bean的声明。或者你可以在使用AspectJ风格的切入点应用安全到整个服务层的多个Bean类。

`<global-method-security>` 元素

这个元素用来在你的应用程序中启用基于注解的安全性（通过设置元素的 `appropriate` 属性），同时分组将应用到整个应用程序上下文的安全切入点声明。你应该只定义一个 `<global-method-security>` 元素。下面的定义可以开启Spring Security的 `@Secured` 支持：

```
<global-method-security secured-annotations="enabled" />
```

添加一个注解到类或者接口的方法中可以限制对相应方法的访问。Spring Security的原生注解支持定义了一套用于该方法的属性。这些将被传递到 `AccessDecisionManager` 用来做实际的决定：

```
public interface BankService {

@Secured("IS_AUTHENTICATED_ANONYMOUSLY")
public Account readAccount(Long id);

@Secured("IS_AUTHENTICATED_ANONYMOUSLY")
public Account[] findAccounts();

@Secured("ROLE_TELLER")
public Account post(Account account, double amount);
}
```

启用 JSR-250 注解使用

```
<global-method-security jsr250-annotations="enabled" />
```

这些都是基于标准的，并允许应用简单的基于角色的约束，但是没有Spring Security的原生注解强大。要使用新的基于表达式的语法，你可以使用

```
<global-method-security pre-post-annotations="enabled" />
```

等价的Java代码如下

```
public interface BankService {

@PreAuthorize("isAnonymous()")
public Account readAccount(Long id);

@PreAuthorize("isAnonymous()")
public Account[] findAccounts();

@PreAuthorize("hasAuthority('ROLE_TELLER')")
public Account post(Account account, double amount);
}
```

基于表达式的注解是一个很好的选择，如果你需要定义超过一个检查当前用户列表中的角色名称的简单的规则。

> 被注解的方法将仅在被定于为Spring 的Bean的实例时才能确保安全（在相同的应用程序的上下文中该方法-启用方法安全检查）。如果你想确保非Spring创建的实例的安全性（比如使用new操作符创建的），那么你需要使用AspectJ。
>
> 你可以在同一个应用程序中启用多种注解，但是在一个接口或者类中只能使用一种类型的注解，否则会出现不明确的行为。如果对特定的方法使用了两个注解，只有其中的一个会被应用。

使用 `protect-pointcut` 添加切入点保护

```xml
```

这将保护在应用程序上下文定义的所有在 `com.mycompany` 包下类名以"Service"结尾的类的方法。只有拥有ROLE_USER角色用户才能执行这些方法。和URL匹配一样，列表中多个匹配的话将会使用第一个匹配的. 安全注解比切入店有更高的优先级。

默认的AccessDecisionManager

这一章节假设你有一些Spring Security中访问控制的底层架构的知识。如果没有，你可以跳过他，后面再来看，这部分针对那些真正需要进行一些定制而不是简单的基于角色的安全的用户。

当你使用命名空间配置时，一个 `AccessDecisionManager` 实例将会被自动创建并注册用来按照你在 `intercept-url` 和 `protect-pointcut` （还有如果你使用了方法注解安全也包含在内）定义的访问属性进行访问决策。

默认的策略是使用一个 `AffirmativeBased` `AccessDecisionManager` 和一个 `RoleVoter` 和一个 `AuthenticatedVoter` ,你可以找到从验证章节找到更多信息。

定制AccessDecisionManager

如果你需要使用一个更复杂的访问控制策略，那么很容易的为方法和Web安全设置替代方案。

对于方法安全，通过在 `global-method-security` 上设置 `access-decision-manager-ref` 属性来为应用程序指导适当的 `AccessDecisionManager` 的Bean ID.

```
<global-method-security access-decision-manager-ref="myAccessDecisionManagerBean">
...
</global-method-security>
```

网站安全的语法一样，但是是在 `http` 元素上。

```
<http access-decision-manager-ref="myAccessDecisionManagerBean">
...
</http>
```

验证管理器和命名空间

Spring Security中主要的提高验证服务的接口是`AuthenticationManager`,这通常是一个Spring Security的`ProviderManager`类的实例。如果你以前用过框架你可能已经熟悉了。如果不是后面的技术预览章节会讲到。这个bean是通过`authentication-manager`命名空间来注册。你不能使用自定义的AuthenticationManager如果通过命名空间使用HTTP或方法安全，但是这不应该是一个问题，因为你可以完全控制所使用的好的AuthenticationProvider。

您可能需要使用`ProviderManager`注册其他的`AuthenticationProvider` Bean，你可以使用`<authentication-provider>`元素的`ref`属性，在属性的值是您要添加的bean的名字。例如：

```
<authentication-manager>
<authentication-provider ref="casAuthenticationProvider"/>
</authentication-manager>

<bean id="casAuthenticationProvider"
    class="org.springframework.security.cas.authentication.CasAuthenticationProvider">
...
</bean>
```

另一个常见的需求是，上下文中的另一个bean可能需要引用AuthenticationManager，您可以注册一个AuthenticationManager的别名在应用程序上下文中的其他地方使用这个名字。

```
<security:authentication-manager alias="authenticationManager">
...
</security:authentication-manager>

<bean id="customizedFormLoginFilter"
    class="com.somecompany.security.web.CustomFormLoginFilter">
<property name="authenticationManager" ref="authenticationManager"/>
...
</bean>
```

应用程序示例

这里有几个可用的网站应用程序示例。为了避免大量的下载。只有"totorial"和"contacts"示例包含到了分发的zip文件。其他的可用按照介绍的描述从源代码构建。你可用很容易的自己构建项，通过http://spring.io/spring-security/网站可用获取更多信息。本章中提到的所有路径是相对于项目的源目录。

Tutorial 示例

本教程样本是一个让你入门的很好的基础的例子。它完全使用了简单命名空间配置。编译好的应用程序包含在分发的zip包中。随时可以部署到你的Web容器（spring-security-samples-tutorial-3.1.x.war）。基于表单的认证机制结合记住我功能，验证供应商提供使用cookie自动记住登录。

我们建议您从本示例开始，因为XML非常小，易于遵循。最重要的是，你可以把这个XML文件（和它对应的web.xml入口）轻松地添加到现有的应用程序。在这个基本集成成功的时候，我们建议你试着添加方法验证和领域对象安全。

Contacts 示例

该示例是一个高级的例子，它展示除了基本的应用程序安全领域对象的访问控制列表（ACL）的更强大的功能。本申请提供了一个接口让用户能够管理简单的联系人数据库（域对象）。

拷贝WAR文件从Spring Security 分发包到你自己的容器的webapps目录来部署它。war名字是spring-security-samples-contacts-3.1.x.war （扩展的版本号取决于你的版本号）

开始你的容器后检查应用程序是否可用载入，访问 http://localhost:8080/contacts (或者其他适合你的容器的URL)

接下来，单击“Debug”。系统将提示您进行身份验证，以及页面提示的一系列用户名和密码。简单验证，显示一个结果页面。它应该包含类似于以下成功消息：

```
Security Debug Information

Authentication object is of type:
org.springframework.security.authentication.UsernamePasswordAuthenticationToken

Authentication object as a String:

org.springframework.security.authentication.UsernamePasswordAuthenticationToken@1f127853:
Principal: org.springframework.security.core.userdetails.User@b07ed00: Username: rod; \
Password: [PROTECTED]; Enabled: true; AccountNonExpired: true;
credentialsNonExpired: true; AccountNonLocked: true; \
Granted Authorities: ROLE_SUPERVISOR, ROLE_USER; \
Password: [PROTECTED]; Authenticated: true; \
Details: org.springframework.security.web.authentication.WebAuthenticationDetails@0: \
RemoteIpAddress: 127.0.0.1; SessionId: 8fkp8t83ohar; \
Granted Authorities: ROLE_SUPERVISOR, ROLE_USER

Authentication object holds the following granted authorities:

ROLE_SUPERVISOR (getAuthority(): ROLE_SUPERVISOR)
ROLE_USER (getAuthority(): ROLE_USER)

Success! Your web filters appear to be properly configured!
```

一旦你成功地收到上述消息，返回到示例应用程序的主页，点击“Manage”。然后，您可以使用这个应用程序。注意只有当前用户的联系人会被显示，并且只有拥有 `ROLE_SUPERVISOR` 可用授权去删除联系人，在幕后 `MethodSecurityInterceptor` 在保护业务对象。

该应用程序允许您修改与不同的联系人相关联的访问控制列表。一定要试试这个，并了解它是如何工作通过检查应用程序上下文XML文件。

LDAP 示例

LDAP例子程序提供了一个基本的配置和使用命名空间和使用传统的bean同等设置在同一个应用程序上下文文件。这意味着，其实是在这个应用程序中配置了两个相同的身份验证提供者。

OpenID 示例

OpenID的示例演示了如何使用命名空间来配置OpenID和如何设置为Google，Yahoo和M OpenID设置属性交换配置（您可以添加其他的供应商，如果你愿意的话）。它使用一个基于JQuery的openid选择框，以提供用户友好的登录页面，它允许用户很容易地选择一个验证提供者，而不是输入OpenID标识符。

应用程序的不同之处在于它允许任何用户访问该站点正常认证情景（只要它们的OpenID身份验证是成功的）。当您第一次登录时，你会得到一个“Welcome [your name]”“的消息。如果您注销并重新登录（以相同OpenID身份），那么这应更改为”欢迎回来“，这是通过使用一个实现自定义的 `UserDetailsService` 它分配一个标准角色给的任何用户，并在内部存储的身份的map。显然，一个真正的应用程序将使用一个数据库替换他。看一看源代码了解更多信息，此类还考虑到，不同供应商返回不同属性，并建立相应用户名。

CAS 示例

CAS示例要求您同时运行CAS服务器和CAS客户端。它不包含在发行包，所以你应该按照描述签出目代码。你会发现sample/cas目录下的相关文件。还有在那里一个readme.txt文件，这也解释了如何直接从源代码运行服务器和客户端，完全支持SSL。

JAAS 示例

该JAAS是很简单的如何用Spring Security使用JAAS LoginModule的例子。所提供的 `LoginModule` 将成功地验证用户，如果用户名和密码相等，否则抛出一个 `LoginException` 异常。在本实例中使用的的 `AuthorityGranter` 总是授予角色ROLE*USER*。示例应用程序还演示了如何通过设置 *`LoginModule`* 的 *`jaas-api-provision`* 等于 *`true`* 返回*JAAS Subject* 来运行*(**)*。

Pre-Authentication 示例

此示例应用程序演示了如何绑定bean从预认证框架从Java EE容器使用的登录信息。用户名和角色是由容器设置的。

代码是 samples/preauth.

Spring Security 社区

问题跟踪

Spring Security使用JIRA来管理bug报告和改进请求。如果您发现错误，请使用JIRA记录报告。不要在支持论坛，邮件列表，或通过电子邮件项目的记录。这些方法是临时的，我们更喜欢使用正式的流程。

如果可能的话，在你的问题的报告，请提供一个JUnit测试，演示任何不正确的行为。或者，更好的是，提供了一个可以解决这个问题补丁。同样，非常欢迎的提出改进需求，虽然我们只接受、有对应的单元测试的改进请求。这是必要的，以确保保持项目的测试覆盖率。

问题跟踪系统你可用通过这里访问 http://jira.springsource.org/browse/SEC.

成为参与者

我们欢迎你参与Spring Security项目。有贡献的方法很多，包括阅读论坛，并响应来自其他人的问题，编写新的代码，改进现有的代码，协助文档，开发样品或教程，或者干脆提出建议。

更多信息

欢迎为Spring Security提出问题和意见。您可以使用Spring社区论坛网站http://forum.springsource.org和其他Spring Security框架的用户进行讨论。请记住，正如前面说的使用JIRA 提交bug报告。

架构和实现

一旦你熟悉了如何设置和运行一些基于命名空间的配置的应用程序，你可能希望了解在命名空间后面这个框架实际的是如何工作的。像大多数软件一样，Spring Security有一些重要的接口和类和一些贯穿整个框架的概念。在参考指南的这部分，我们将看看其中的一部分，看看他们如何协同工作来为Spring Security提供身份验证和访问控制的支持。

技术预览

运行时环境

Spring Security 3.0 需要Java 5.0或者更高版本的运行环境。由于Spring Security是自包含的方式进行操作，所以不需要放置特别的配置文件到你的Java运行环境。特别的，不需要配专门的Java认证和授权服务(JAAS)策略文件或者放到Spring Security普通的类路径下。

同样如果你使用EJB容器或者Servlet容器，没有必要放置配置文件到任何地方，也不需要包含Spring Security到服务器的类加载器。所有必须的文件将包含到你的应用程序中。

这种设置提供了部署时最大的灵活性。你可以简单的复制你的目标文件(可能是JAR,WAR,或者EAR)从一个系统到另一个系统。它会立即开始工作。

核心组件

In Spring Security 3.0, the contents of the spring-security-core jar were stripped down to the bare minimum. It no longer contains any code related to web-application security, LDAP or namespace configuration. We'll take a look here at some of the Java types that you'll find in the core module. They represent the building blocks of the the framework, so if you ever need to go beyond a simple namespace configuration then it's important that you understand what they are, even if you don't actually need to interact with them directly.

Web应用安全

大部分的Spring Security用户都会使用到他们的HTTP或则Servlet API应用中。在这一部分我们将了解Spring Security如何在应用的web层提供验证和权限控制。我们将会看到在命名空间的背后实际上是哪些类和接口被装配起来用来提高web层的安全。在某些情况下，有必要使用传统的Bean的配置，提供完全的配置控制，因此我们也将看到如何直接配置这些没有命名空间的类。